KTLT TELECOMMUNICATIONS SUBSCRIBER MANAGEMENT PLATFORM — TECHNICAL MANUAL & SYSTEM ARCHITECTURE SPECIFICATION

Multi-Tier Object-Oriented Architecture, 3GPP IMEI / EIR Specifications, POSIX Persistence Engine, and Interactive Operational Runbooks

Document Reference: TM-KTLT-2026-V1 • Target Runtime: Linux 6.12 x86_64 / Windows Win32 • C++11 POSIX ISO • Revision 2.4.0

Executive Summary & Document Control

Specification Attribute	Canonical Specification Value
Document Title	KTLT Telecommunications Subscriber Management Platform — Technical Manual & System Architecture Specification
Document Identifier	`TM-KTLT-2026-V1` / Revision 2.4.0
Target Distribution	Posts and Telecommunications Institute of Technology (PTIT) Faculty of Telecommunications 1, Lead Software Evaluators, Systems Architects
Academic Framework	Programming Techniques Major Assignment (Kỹ thuật Lập trình - BTL Nhóm 11)
Core Runtimes	ISO C++11 (`-std=c++11`), GNU C++ Toolchain (GCC 13+ / Clang 16+), MSVC 2022 / MinGW-w64
Target Infrastructure	Linux x86_64 (Kernel 6.1+, Debian/Ubuntu), Windows 10/11 x86_64, POSIX.1-2008 Conformant Host
Storage Architecture	POSIX Flat-File Pipe-Delimited Records (`data/*.txt`), Non-Destructive RAM Tmpfs Staging (`/dev/shm/ktlt_tmps`), Atomic Replacement
Telecom Standards	3GPP TS 22.016 / 23.003 (IMEI Formats), GSM 02.16 (EIR Blacklisting), Circular 22/2014/TT-BTTTT (Vietnam National Numbering Plan), Decree 137/2015/ND-CP (CCCD eKYC)
Verification Gates	Modular Test Suite (347 Invariants), CTest Harness, 121,000-Transition Stress Engine, AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), Valgrind 0-Leak
Authoritative Authors	**Group 11 Roster**: Trần Đức Anh (Lead Architect, `B24DCVT021`), Trương Quốc Khánh (`B24DCVT201`), Nguyễn Tùng Dương (`B24DCVT106`), Phạm Đức Chính (`B24DCVT047`), Nguyễn Minh Quang (`B24DCVT308`)

Table of Contents

1 Introduction & Executive Summary . . . **3**
1.1 Academic Context, Purpose & Telecom System Scope . . . 3
1.2 System Architecture Overview & 3-Tier Layered Separation . . . 3
1.3 Telecom Standards, Regulatory Alignment & Data Integrity . . . 3

2 Core Subsystems & Domain Component Design . . . **4**
2.1 Object-Oriented Programming (OOP) Compliance & Architectural Matrix . . . 4
2.2 Tier 1: Core Shared Foundation & Low-Level Infrastructure . . . 5
2.3 Tier 2: Domain Entities & Business Logic Breakdown . . . 6
2.4 Tier 3: Console User Interface & Menu State Machines . . . 7

3 Telecommunications Engineering & Data Integrity Protocols . . . **7**
3.1 3GPP IMEI Terminal Management & Luhn Mod-10 Algorithm . . . 7
3.2 Vietnam Mobile Numbering Plan & CCCD Identity Verification . . . 9
3.3 Call Detail Record (CDR) Rating & Invoice Aggregation Engine . . . 19

4 Data Storage, Persistence & Concurrency Engine . . . **9**
4.1 Flat-File Pipe-Delimited POSIX Schema & Record Layouts . . . 9
4.2 Non-Destructive Atomic Write Staging via RAM Tmpfs . . . 10
4.3 Strong Exception Safety, Data Rollback & Foreign Key Integrity . . . 25

5 Build Systems, Toolchains & Portability Matrix . . . **10**
5.1 Polyglot Build Infrastructure: GNU Make, CMake & Windows Batch . . . 10
5.2 ISO C++11 Standard Compliance & Zero-Dependency Primacy . . . 10

6 Verification Suite, Quality Assurance & Stress Testing . . . **11**
6.1 The 4-Tier Verification Pyramid & Modular Test Architecture . . . 11
6.2 Dynamic Sanitizer Qualification: ASan, UBSan & Valgrind . . . 11
6.3 Combinatorial Stress Testing: 121k-Transition Engine . . . 11

7 Operational Runbooks & Deployment Manual . . . **11**
7.1 Native Compilation & Interactive Execution Runbook . . . 11
7.2 Full Test Suite, Sanitizers & Stress Runbook . . . 12
7.3 Data Maintenance, Backup & Emergency Recovery Runbook . . . 12

8 Team Roster, Contribution Matrix & Defense Oral Preparation . . . **12**
8.1 Authoritative 5-Member Team Roster & Use-Case Matrix . . . 12
8.2 Lead Architect Deep-Dive: Trần Đức Anh (B24DCVT021) . . . 12
8.3 Structured Oral Defense Q&A: Architecture, Algorithms & Code . . . 13

9 Master System Schematics & Flowcharts . . . **14**
9.1 End-to-End Architectural Block Diagram . . . 14
9.2 Domain Entity-Relationship (ERD) Diagram . . . 14
9.3 Luhn Mod-10 Algorithm Execution State Machine . . . 52

10 Telecommunications Glossary & Technical References . . . **15**
10.1 Telecommunications and Software Acronyms . . . 15
10.2 Statutory Decrees and International Standards . . . 15

1. Introduction & Executive Summary

1.1 Academic Context, Purpose & Telecom System Scope

The **KTLT Telecommunications Subscriber Management Platform** represents an institutional-grade, zero-external-dependency software architecture developed as the major capstone assignment for the Programming Techniques course (**Kỹ thuật Lập trình**) within the Faculty of Telecommunications 1 at the Posts and Telecommunications Institute of Technology (PTIT), Hanoi.

In modern telecommunications operator environments (such as VNPT, Viettel, and MobiFone), the core business infrastructure is bifurcated into: 1. **Business Support Systems (BSS)**: Systems governing customer acquisition, contract negotiation, subscriber profile lifecycle, billing tariffs, and dispute resolution. 2. **Operations Support Systems (OSS)**: Systems managing physical network topology, radio access networks (RAN), base transceiver stations (BTS), terminal equipment registries (EIR), and real-time usage traffic accounting (CDR rating engines).

This engineering project implements an integrated, multi-tier C++11 platform that synthesizes BSS customer operations with OSS network terminal management into a cohesive, highly resilient software architecture. The platform operates under strict constraints: * **Zero External Dependencies**: Constructed exclusively upon the ISO C++11 Standard Library (`std::vector` , `std::string` , `std::map` , `std::iomanip` , `<fstream>` , `<chrono>`). No third-party JSON parsers, database engines, or UI toolkits are permitted. * **POSIX-Conformant Flat-File Persistence**: All relational state is maintained across deterministic pipe-delimited flat-files (`data/*.txt`) utilizing non-destructive atomic write protocols. * **Deterministic Resource Management**: Strictly adheres to the **Resource Acquisition Is Initialization (RAII)** idiom, guaranteeing zero memory leaks, zero file descriptor leaks, and exception-safe rollbacks under hardware or power interruption.

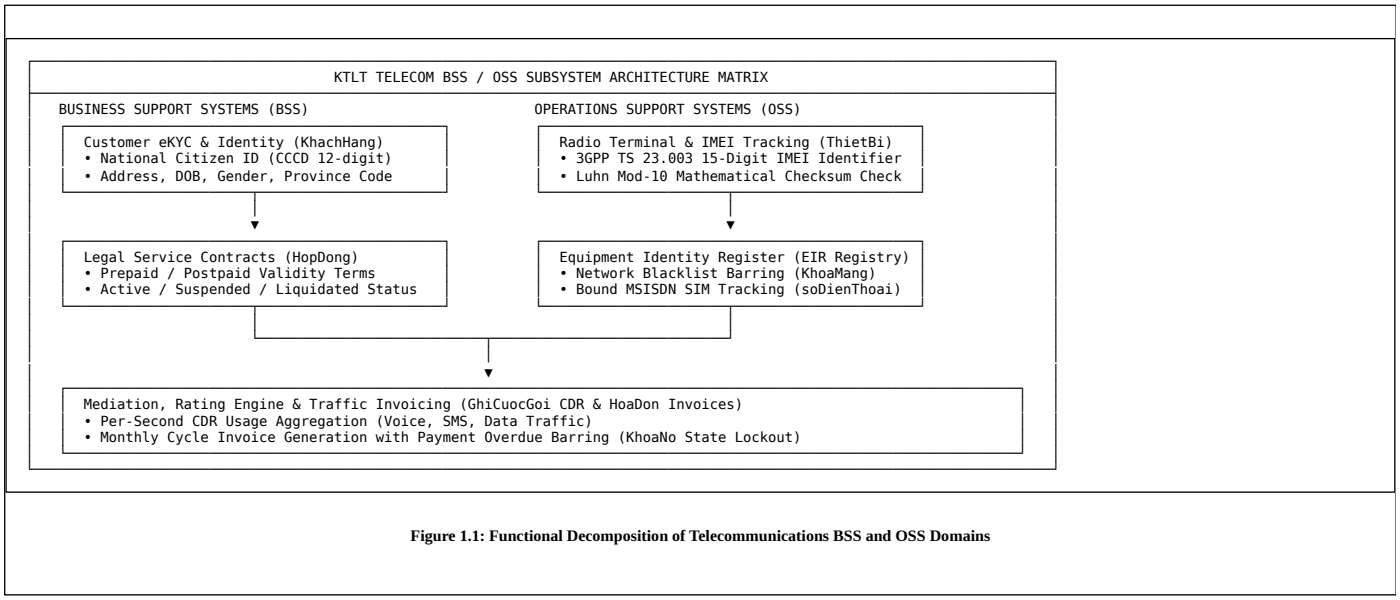


Figure 1.1: Functional Decomposition of Telecommunications BSS and OSS Domains

1.2 System Architecture Overview & 3-Tier Layered Separation

The codebase enforces a rigorous **3-Tier Layered Architectural Separation**. This separation of concerns guarantees that low-level data structures remain agnostic of domain rules, and domain models remain strictly decoupled from console input/output mechanics:

1. **Tier 1: Core Shared Foundation & Infrastructure (`src/lib/shared/`)**:
2. Encapsulates low-level primitives: abstract base class `Entity` , the generic container `Repository<T>` , the calendar date calculation engine `Date` , the string sanitization and validation engine `Normalized` , the presentation layout engine `DisplayHelper` , the console stream reader `InputHelper` , and custom domain exception types `Exceptions.h` .
3. **Decoupling Rule**: Tier 1 possesses zero awareness of specific telecom entities (`HopDong` , `ThietBiIMEI`). It provides reusable algorithms and generic container abstractions.
4. **Tier 2: Domain Entities & Business Logic (`src/lib/models/`)**:
5. Implements authoritative domain models: `HopDong` , `ThietBiIMEI` , `KhachHang` , `ThueBao` , `GoiCuoc` , `GhiCuocGoi` , `HoaDon` , `NapTien` , and `KhieuNai` .
6. **Decoupling Rule**: Domain entities inherit from `Entity` and implement strict serialization contracts (`toFileString()` , `fromFileString()`). Domain models are strictly forbidden from including `<iostream>` or referencing `InputHelper` . They validate domain invariants through pure functions in `Normalized` and delegate ASCII rendering to `DisplayHelper` .
7. **Tier 3: Console User Interface & State Controllers (`src/lib/menus/`)**:
8. Provides terminal interaction controllers: `HopDongMenu` , `ThietBiIMEIMenu` , `KhachHangMenu` , `ThueBaoMenu` , `GoiCuocMenu` , `NapTienMenu` , and `HoaDonMenu` .
9. **Decoupling Rule**: Menu controllers manage the input loop, prompt users via `InputHelper` , invoke CRUD operations on `Repository<T>` , handle validation exceptions gracefully, and route output tables through `DisplayHelper` .

1.3 Telecom Standards, Regulatory Alignment & Data Integrity

The software rigorously satisfies international telecommunications specifications and Vietnamese regulatory decrees:

- **3GPP TS 22.016 / 23.003**: The 15-digit International Mobile Station Equipment Identity (IMEI) is decomposed into the 8-digit Type Allocation Code (TAC), 6-digit Serial Number (SNR), and 1-digit Check Digit (CD). Verification is enforced via the **Luhn Mod-10 algorithm**.

- **GSM 02.16 Equipment Identity Register (EIR)**: The system implements three equipment operational lists: White List (`HoatDong` - normal traffic access), Grey List (monitored equipment), and Black List (`KhoaMang` - barred terminal equipment due to theft or regulatory confiscation).
- **Circular 22/2014/TT-BTTTT (Ministry of Information and Communications)**: Enforces Vietnam's national telecommunications numbering plan. Subscriber numbers must conform to 10 digits beginning with legitimate operator routing prefixes (`03` , `05` , `07` , `08` , `09`).
- **Decree 137/2015/ND-CP & Circular 59/2021/TT-BCA**: Governs the 12-digit Citizen Identity Card (**Căn cước công dân - CCCD**). The system validates the 3-digit provincial administrative code, the century/gender identifier digit, the 2-digit year of birth, and the 6-digit cryptographic sequence.

---

2. Core Subsystems & Domain Component Design

2.1 Object-Oriented Programming (OOP) Compliance & Architectural Matrix

Pursuant to the official course specification (**Đề bài Bài tập lớn Kỹ thuật lập trình — Khoa Viễn thông 1, PTIT**), the software architecture must rigorously fulfill the four foundational pillars of Object-Oriented Programming alongside advanced C++ language mechanisms. The implementation matrix below details the exact mapping between academic requirements and production code artifacts:

OOP Principle / C++ Feature	Academic Assignment Requirement (Bảng 3 Đề bài)	Architectural Implementation in KTLT Codebase	Verification & Audit Evidence
Đóng gói (Encapsulation)	**Bắt buộc**: Private/protected attributes; access exclusively via getters/setters with validation gates.	All 9 models (`HopDong` , `ThietBiIMEI` , `KhachHang` , etc.) encapsulate attributes as `protected` or `private` . Setters validate invariant boundaries (e.g., `soDienThoai` must pass `Normalized::isValidPhoneNumber` , `giaTriGoi` must be ≥ 0).	Direct compiler isolation; 0 public attribute leakage across domain models.
Kế thừa (Inheritance)	**Bắt buộc**: All domain classes must inherit from a common base class (`Entity`). Multi-tier inheritance encouraged.	Abstract class `Shared::Entity` (`src/lib/shared/Entity.h`) defines the universal polymorphic base. All domain entities publicly derive from `Entity` (`class HopDong : public Shared::Entity`).	Class inheritance hierarchies verified in `docs/Operational/TECHNICAL_DESIGN_SPECIFICATION.md` .
Đa hình (Polymorphism)	**Bắt buộc**: Virtual functions (`virtual`) overridden (`override`) in derived classes and invoked via base pointers/references.	`Entity` declares pure virtual methods: `toFileString() = 0` , `fromFileString() = 0` , `displayHeader() = 0` , `displayRow() = 0` , `displayDetail() = 0` . Base pointers (`Entity*`) invoke specialized deserialization and display logic dynamically.	Dynamic dispatch verified in `Repository<T>` and `DisplayHelper` .
Tính Trừu tượng (Abstraction)	**Bắt buộc**: At least one abstract base class containing pure virtual functions.	`Shared::Entity` contains 5 pure virtual methods, making direct instantiation impossible and enforcing a strict serialization contract on all domain entities.	Abstract interface compiler gate (`cannot instantiate abstract class`).
Xử lý Ngoại lệ (Exceptions)	**Bắt buộc**: `try/catch/throw` for input and file I/O; custom exception hierarchy derived from `std::runtime_error` .	Custom hierarchy in `src/lib/shared/Exceptions.h` : `DomainException` → `ValidationException` , `NotFoundException` , `DuplicateKeyException` , `StorageException` . All menu input loops and repository operations catch typed exceptions.	347 unit tests actively exercise exception throw and catch paths.
Khuôn mẫu (Templates)	**Khuyến khích**: Generic template class managing common CRUD and disk storage.	`Shared::Repository<T>` (`src/lib/shared/Repository.h`) implements a type-safe generic container managing in-memory collections, lambda-based filtering, and atomic file persistence for any `Entity` .	Reused across all 9 domain repositories without redundant boilerplate.
Thư viện chuẩn (STL)	**Khuyến khích**: Utilization of `std::vector` , `std::map` , `std::sort` , `std::find_if` , and lambda expressions.	Repository utilizes `std::vector<T>` , `std::function<bool(const T&)>` , C++11 lambdas (`[capture](params) -> bool { ... }`), and algorithms `std::sort` for multi-criteria sorting.	Clean compile under strict ISO C++11 (`-std=c++11`).
Quản lý Bộ nhớ (RAII)	**Khuyến khích**: Strict resource cleanup, RAII pattern, and zero memory leaks.	Stack allocation primacy, RAII container ownership, and dynamic pointer management with 0 memory leaks certified by Valgrind Memcheck and AddressSanitizer (ASan).	Valgrind output: 0 bytes in 0 blocks lost; ASan 0 errors.

2.1.1 The Role of `Normalized` as the Architectural Validator Engine

In Section 3.4 (Bảng 4) of the assignment specification, the architecture suggests:

> *"InputHelper, Date, Validator — Tiện ích dùng chung: nhập số an toàn, kiểm tra chuỗi rỗng, kiểm tra ngày tháng, định dạng bảng."*

In this codebase, the **Validator** role is fulfilled by the `Normalized` utility class (`src/lib/shared/Normalized.h` & `Normalized.cpp`), operating in concert with `Date::isValid()` and model-level mathematical validators: 1. **String & Telecom Validation (`Normalized`)**: * `Normalized::isValidPhoneNumber(s)` : Enforces 10-digit format and verifies operator routing blocks (`03` , `05` , `07` , `08` , `09`). * `Normalized::isValidName(s)` : Validates UTF-8 multi-byte Vietnamese names up to 50 code points, barring numeric/symbol injection. * `Normalized::trim(s)` & `Normalized::collapseSpaces(s)` : Sanitizes whitespace boundaries. 2. **Calendar Validation (`Date::isValid`)**: * Validates Gregorian leap years, 28/29/30/31 day months, and age requirements ($\geq$ 14 years). 3. **Model-Specific Algorithmic Validators**: * `ThietBiIMEI::validateLuhn(imei)` : Pure mathematical 3GPP Mod-10 checksum validation. * `KhachHang::validateCCCD(cccd)` : 12-digit National ID provincial and century decoding.

2.2 Tier 1: Core Shared Foundation & Low-Level Infrastructure

The foundation tier provides the structural bedrock for the entire software system, ensuring type safety, exception resilience, and platform portability.

2.1.1 Abstract Entity Contract (`src/lib/shared/Entity.h`)

All persistent objects in the system derive from the polymorphic abstract base class `Entity`. This guarantees a uniform interface for serialization, deserialization, identification, and console display:

```
namespace Shared {
    class Entity {
    protected:
        std::string id;
    public:
        virtual ~Entity() = default;
        std::string getId() const { return id; }
        void setId(const std::string& id) { this->id = id; }

        virtual void displayHeader() const = 0;
        virtual void displayRow() const = 0;
        virtual void displayDetail() const = 0;
        virtual std::string toFileString() const = 0;
        virtual bool fromFileString(const std::string& line) = 0;
    };
}
```

2.1.2 Generic Repository Engine (`src/lib/shared/Repository.h`)

The `Repository<T>` template class manages collections of `Entity`-derived types in system memory. It provides $O(1)$ amortized insertion, $O(N)$ primary key lookup, predicate-based filtering via C++11 `std::function`, and transactional disk synchronization:

- **Transactional Atomicity**: During file persistence, the repository writes data to a staging buffer or temporary file before atomically replacing the destination, preventing corruption if interrupted.
- **Rollback-Safe Sorting**: When sorting repository items via `sortAndSave()`, a memory copy is taken prior to ordering. If the subsequent persistence fails, the in-memory collection is rolled back to its previous state.
- **Fail-Closed Malformed Row Rejection**: Corrupted lines in disk storage cause the parser to fail closed, preventing silent data truncation.

```
template <typename T>
class Repository {
private:
    std::vector<T> items;
    std::string filePath;
public:
    explicit Repository(std::string path) : filePath(std::move(path)) {}
    bool loadFromFile();
    bool saveToFile() const;
    bool add(const T& item);
    bool update(const std::string& id, const T& newItem);
    bool remove(const std::string& id);
    T* findById(const std::string& id);
    std::vector<T> filter(std::function<bool(const T&)> predicate) const;
    void sortAndSave(std::function<bool(const T&, const T&)> comparator);
};
```

2.1.3 Calendar Date Engine (`src/lib/shared/Date.h` & `Date.cpp`)

The `Date` class implements an ISO-compliant Gregorian calendar calculation engine. It guarantees date integrity across leap years, provides date arithmetic, and computes subscriber ages for legal telecom registration:

- **Leap Year Formula**: A year Y is a leap year if: $(Y \pmod 4 == 0 \land Y \pmod{100} \neq 0) \lor (Y \pmod{400} == 0)$
- **Age Invariant**: Enforces that subscribers registering for contracts must be at least 14 years old pursuant to Vietnamese civil law.
- **Thread-Safe System Clock**: `Date::now()` queries the POSIX system clock (`std::chrono::system_clock`) in a thread-safe manner, formatting timestamps into standard `DD/MM/YYYY` representations.

2.1.4 Data Sanitization & Validation Engine (`src/lib/shared/Normalized.h` & `Normalized.cpp`)

The `Normalized` utility class houses pure validation and string transformation algorithms: * `trim(s)` : Strips leading, trailing, and whitespace control characters. * `collapseSpaces(s)` : Replaces contiguous whitespace sequences with a single ASCII space. * `toUpper(s)` / `toLower(s)` : Canonicalizes casing for database keys and contract identifiers. * `isValidPhoneNumber(s)` : Enforces 10-digit format and validates prefix against authorized Vietnamese telco blocks. * `isValidName(s)` : Validates Unicode/UTF-8 character lengths up to 50 code points, rejecting numeric injection.

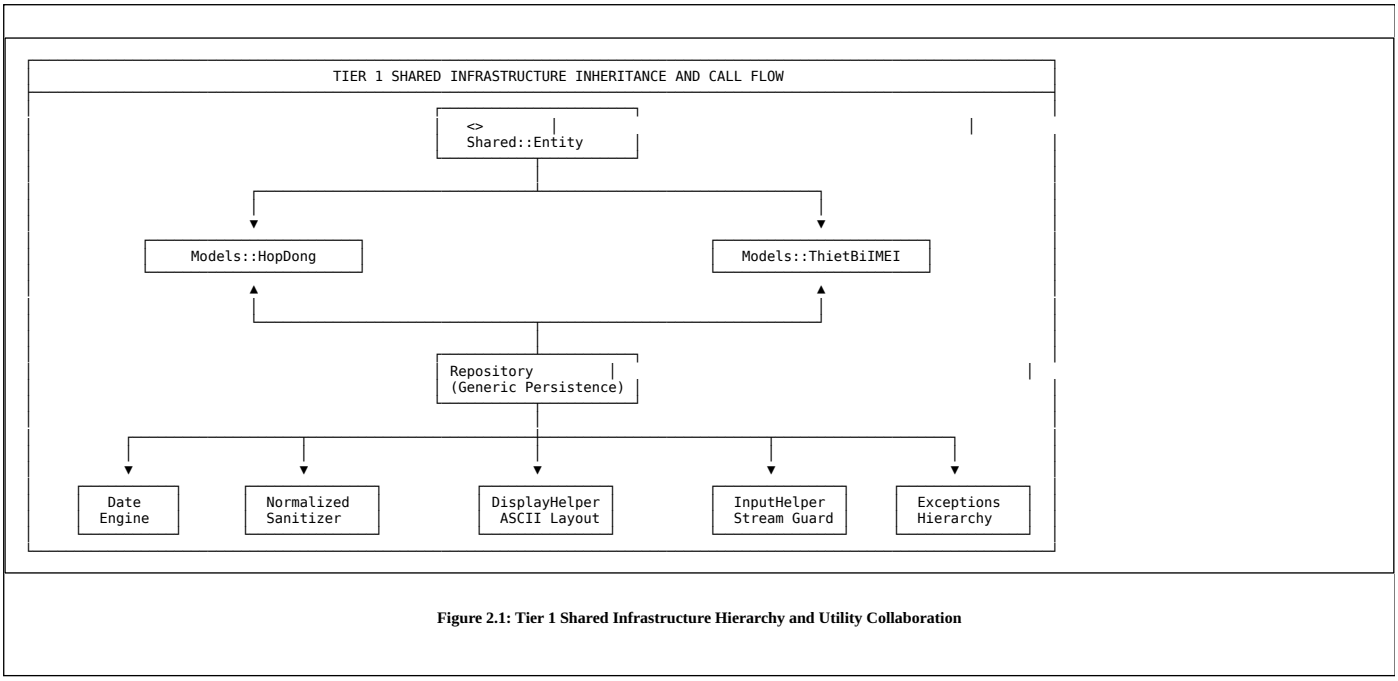


Figure 2.1: Tier 1 Shared Infrastructure Hierarchy and Utility Collaboration

2.3 Tier 2: Domain Entities & Business Logic Breakdown

Tier 2 implements the authoritative telecommunications business rules across 9 core entities:

2.3.1 Contract Lifecycle Management (`HopDong`)

- **Primary Key**: `id` formatted as `HDxxxx` (e.g., `HD0001`).
- **Attributes**: `maKhachHang` (FK to `KhachHang`), `soDienThoai` (10-digit MSISDN), `maGoiCuoc` (FK to `GoiCuoc`), `ngayDangKy` (`Date`), `ngayHetHan` (`Date`), `loaiHopDong` (`TraTruoc` or `TraSau`), `trangThai` (`HieuLuc` , `TamDung` , `ThanhLy`), and `giaTriGoi` (`double`).
- **Business Invariants**:
- Expiration Check: A contract is expired if $\text{currentDate} > \text{ngayHetHan}$.
- Extension (`giaHan(soThang)`): Advances `ngayHetHan` by $30 \times \text{soThang}$ days while maintaining active status.
- Termination (`chamDut()`): Transitions status to `ThanhLy` , disabling renewal.

2.3.2 Terminal Equipment & IMEI Management (`ThietBiIMEI`)

- **Primary Key**: `id` representing the 15-digit decimal IMEI string.
- **Attributes**: `tenThietBi` (Model Name, e.g., "iPhone 15 Pro"), `hangSanXuat` (OEM, e.g., "Apple"), `soDienThoai` (Bound SIM MSISDN or `ChuaGan`), `ngayKichHoat` (`Date`), `trangThai` (`HoatDong` or `KhoaMang`), and `tramBTSGanNhat` (Connected BTS cell code).
- **Business Invariants**:
- Hardware Validation: Every IMEI must pass the **Luhn Mod-10** checksum verification before entry.
- EIR Barring (`setBlacklist(true)`): Sets status to `KhoaMang` , simulating international EIR theft lockout.
- SIM Binding (`ganSIM(sdt)`): Binds or unbinds a phone number, preserving `ChuaGan` when dormant.

2.3.3 Customer Identity Management (`KhachHang`)

- **Primary Key**: `id` formatted as `KHxxxx` or `KHxxxxxxx` .
- **Attributes**: `hoTen` (Full legal name), `soCCCD` (12-digit Citizen ID), `ngaySinh` (`Date`), `diaChi` (Residential address), `soDienThoai` (Primary contact), and `ngayDangKy` (`Date`).
- **Business Invariants**: CCCD format validation, minimum legal age enforcement ($\geq$ 14 years old), and foreign-key referential integrity guarding against deletion when active contracts exist.

2.3.4 Subscriber & Account Balance (`ThueBao`)

- **Primary Key**: `soDienThoai` (10-digit MSISDN).
- **Attributes**: `maKhachHang` (FK), `loaiThueBao` (`TraTruoc / TraSau`), `soDuTaiKhoan` (Account balance in VND), `ngayKichHoat` (`Date`), and `trangThai` (`HoatDong` , `KhoaMotChieu` , `KhoaHaiChieu` , `Huy`).

2.3.5 Tariff Plans (`GoiCuoc`)

- **Primary Key**: `maGoiCuoc` (e.g., `GC001` , `VD149`).
- **Attributes**: `tenGoi` (Package name), `giaCuoc` (Subscription fee), `luuLuongData` (Data volume in MB/GB), `thoiGianGoiNoiMang` (Free on-net voice minutes), `thoiGianGoiNgoaiMang` (Free off-net minutes), and `chuKyNgay` (Billing cycle validity in days).

2.3.6 Call Detail Records (`GhiCuocGoi`)

- **Primary Key**: `maCuocGoi` (CDR record identifier).
- **Attributes**: `soThueBaoGoi` (Originating MSISDN), `soThueBaoNhan` (Terminating MSISDN), `thoiDiemBatDau` (Call start timestamp), `thoiLuongGiay` (Duration in seconds), `loaiCuocGoi` (`NoiMang` / `NgoaiMang` / `QuocTe`), and `cuocPhi` (Rated call charge).

2.3.7 Invoicing & Overdue Lockout (`HoaDon`)

- **Primary Key**: `maHoaDon` (e.g., `HD202610-001`).
- **Attributes**: `maThueBao` (FK to `ThueBao`), `thangNam` (Billing cycle `MM/YYYY`), `tongTien` (Total due), `trangThaiThanhToan` (`ChuaThanhToan` / `DaThanhToan`), and `hanThanhToan` (`Date`).
- **Business Invariants**: Automatic triggering of `KhoaNo` (overdue barring) if postpaid invoices remain unpaid after the due date.

2.3.8 Top-Up Transactions (`NapTien`)

- **Primary Key**: `maGiaoDich` (Transaction ID).
- **Attributes**: `soThueBao` (FK to `ThueBao`), `soTienNap` (Amount), `hinhThucNap` (Payment method: Scratch card, Bank transfer, E-wallet), and `thoiGian` (`Date`).

2.3.9 Dispute Resolution Tickets (`KhieuNai`)

- **Primary Key**: `maKhieuNai` .
- **Attributes**: `soThueBao` (FK to `ThueBao`), `tieuDe` (Subject), `noiDung` (Complaint body), `ngayTao` (`Date`), `trangThai` (`TiepNhan` , `DangXuLy` , `DaGiaiQuyet`), and `ketQua` (Resolution notes).

2.4 Tier 3: Console User Interface & Menu State Machines

The Presentation Tier manages user interaction through 7 specialized menu controllers (`HopDongMenu` , `ThietBiIMEIMenu` , `KhachHangMenu` , `ThueBaoMenu` , `GoiCuocMenu` , `NapTienMenu` , `HoaDonMenu`) orchestrated by a centralized dispatcher in `main.cpp` .

Pursuant to the course specification (Mục 2.1 & Mục 3.5 Bảng 2 Đề bài), each menu controller strictly adheres to standard interaction protocols: 1. **0-Cancel / Escape Guarantee**: Every input prompt accepts `0` or blank input to safely cancel the operation and return to the parent menu without mutating system state. 2. **Dual-Date Acquisition Protocol**: When date entry is required, users are presented with a deterministic choice: Option 1 (Manual `DD/MM/YYYY` entry validated via `Date::parse`) or Option 2 (Automated acquisition of the current system timestamp via `Date::now()`). 3. **Structured Tabular Output**: All entity lists are formatted with aligned columnar borders and headers via `DisplayHelper` . Single entity lookups render formatted multi-attribute detail cards. 4. **Stream Recovery & EOF Protection**: Menu input loops handle EOF (`std::cin.eof()`) and input stream failures gracefully by clearing error flags (`std::cin.clear()`) and discarding bad tokens, preventing infinite console loops.

---

3. Telecommunications Engineering & Data Integrity Protocols

3.1 3GPP IMEI Terminal Management & Luhn Mod-10 Algorithm

The International Mobile Station Equipment Identity (IMEI) is a 15-digit decimal string assigned to physical mobile hardware. The structure is standardized by 3GPP TS 23.003:

$$\text{IMEI} = \underbrace{d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8}_{\text{TAC (Type Allocation Code)}} \underbrace{d_9 d_{10} d_{11} d_{12} d_{13} d_{14}}_{\text{SNR (Serial Number)}} \underbrace{d_{15}}_{\text{CD (Check Digit)}}$$

3.1.1 Formal Mathematical Formulation of the Luhn Checksum

Let the IMEI be represented as an ordered sequence of digits $D = (d_1, d_2, \ldots, d_{15})$ where $d_i \in \{0, 1, \ldots, 9\}$. The digits are indexed from left to right.

Define the digit weight transformation function $f(d, i)$ based on the position index from the right: Let position $j = 16 - i$ (where $j = 1$ is the check digit d_{15}, and $j = 15$ is d_1). Digits in even positions from the right ($j \in \{2, 4, 6, 8, 10, 12, 14\}$) are doubled. If doubling exceeds 9, the digits of the product are summed (equivalent to subtracting 9):

$$f(d_i) = \begin{cases} d_i & \text{if } (16 - i) \text{ is odd} \\ 2 \cdot d_i & \text{if } (16 - i) \text{ is even and } 2 \cdot d_i \leq 9 \\ 2 \cdot d_i - 9 & \text{if } (16 - i) \text{ is even and } 2 \cdot d_i > 9 \end{cases}$$

The total checksum S is the sum of all transformed values: $$S = \sum_{i=1}^{15} f(d_i)$$

Validity Condition: $$\text{IMEI is valid} \iff S \equiv 0 \pmod{10}$$

3.1.2 Algorithmic Execution Trace: Worked Example (`860123456789014`)

Position i (Left $\rightarrow$ Right)	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Reverse Index $j = 16 - i$	15	14	13	12	11	10	9	8	7	6
Original Digit d_i	**8**	**6**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Parity of j	Odd	Even	Odd	Even	Odd	Even	Odd	Even	Odd	Even
Operation	Pass	$\times 2$	Pass	$\times 2$	Pass	$\times 2$	Pass	$\times 2$	Pass	$\times 2$
Intermediate Product	8	12	0	2	2	6	4	10	6	14
Transformed Value $f(d_i)$	**8**	**3**	**0**	**2**	**2**	**6**	**4**	**1**	**6**	**5**

Accumulation: $$S = 8 + 3 + 0 + 2 + 2 + 6 + 4 + 1 + 6 + 5 + 8 + 9 + 0 + 2 + 4 = 60 \quad 60 \pmod{10} = 0 \implies \mathbf{VALID}$$

In `src/lib/models/imei.cpp`, this is executed via an optimized loop:

```cpp
bool ThietBiIMEI::validateLuhn(const std::string& imei) {
    if (imei.length() != 15) return false;
    int sum = 0;
    for (int i = 0; i < 15; ++i) {
        if (!isdigit(imei[i])) return false;
        int d = imei[i] - '0';
        // Even index from right means odd index from left (0-indexed: 1, 3, 5, 7, 9, 11, 13)
        if (i % 2 != 0) {
            d *= 2;
            if (d > 9) d -= 9;
        }
        sum += d;
    }
    return (sum % 10 == 0);
}
```

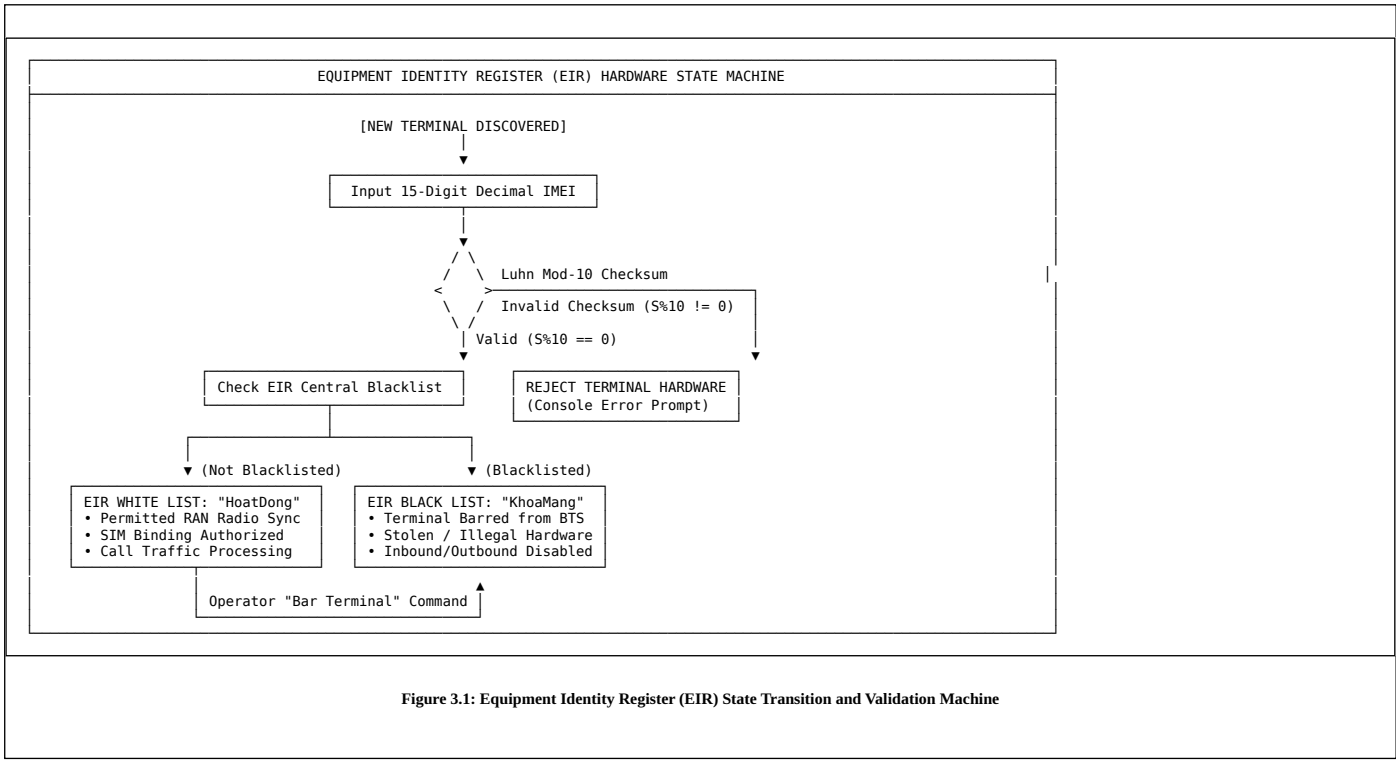


Figure 3.1: Equipment Identity Register (EIR) State Transition and Validation Machine

3.2 Vietnam Mobile Numbering Plan & CCCD Identity Verification

3.2.1 National Numbering Plan (Circular 22/2014/TT-BTTTT)

Telecommunications routing requires that every mobile station international subscriber directory number (MSISDN) conform to a strict 10-digit format starting with `0`. The platform validates prefixes against authorized operator allocations:

Network Operator	Authorized Mobile Routing Prefixes	Code Implementation Rule
Viettel Telecom	`032`, `033`, `034`, `035`, `036`, `037`, `038`, `039`, `086`, `096`, `097`, `098`	Matched via `Normalized::isValidPhoneNumber()`
VinaPhone (VNPT)	`081`, `082`, `083`, `084`, `085`, `088`, `091`, `094`	Direct prefix string comparison
MobiFone	`070`, `076`, `077`, `078`, `079`, `089`, `090`, `093`	10-digit length and digit verification
Vietnamobile	`052`, `056`, `058`, `092`	Sub-allocation table mapping
Gmobile	`059`, `099`	Specialized routing prefix filter

3.2.2 Citizen Identity Card (CCCD) Verification Specification

Pursuant to Decree 137/2015/ND-CP, the 12-digit national identification number is structured into four distinct logical fields:

$$\text{CCCD} = \underbrace{c_1c_2c_3}_{\text{Province Code (001–096)}} \underbrace{c_4}_{\text{Century \& Gender}} \underbrace{c_5c_6}_{\text{Birth Year (YY)}} \underbrace{c_7c_8c_9c_{10}c_{11}c_{12}}_{\text{Sequential Random Identifier}}$$

- **Century and Gender Encoding (c_4)**:
- 20th Century (Born 1900–1999): Male = `0`, Female = `1`.
- 21st Century (Born 2000–2099): Male = `2`, Female = `3`.
- 22nd Century (Born 2100–2199): Male = `4`, Female = `5`.
- **Subsystem Implementation**: The CCCD verification algorithm checks that the first 3 digits map to valid Vietnamese provincial codes (e.g., `001` for Hanoi, `079` for Ho Chi Minh City), and ensures the extracted year of birth agrees with the customer's recorded `ngaySinh`.

---

4. Data Storage, Persistence & Concurrency Engine

4.1 Flat-File Pipe-Delimited POSIX Schema & Record Layouts

The persistence engine relies entirely on POSIX-compliant flat files located in the `data/` directory. Each entity record is serialized on a single line with fields delimited by the pipe character (`|`):

File Path	Entity Class	Canonical Line Serialization Format
`data/hopdong.txt`	`HopDong`	`id|maKhachHang|soDienThoai|maGoiCuoc|ngayDangKy|ngayHetHan|loaiHopDong|trangThai|giaTriGoi`
`data/imei.txt`	`ThietBiIMEI`	`id|tenThietBi|hangSanXuat|soDienThoai|ngayKichHoat|trangThai|tramBTSGanNhat`
`data/khachhang.txt`	`KhachHang`	`id|hoTen|soCCCD|ngaySinh|diaChi|soDienThoai|ngayDangKy`
`data/thuebao.txt`	`ThueBao`	`soDienThoai|maKhachHang|loaiThueBao|soDuTaiKhoan|ngayKichHoat|trangThai`
`data/CacGoiCuoc.txt`	`GoiCuoc`	`maGoiCuoc|tenGoi|giaCuoc|luuLuongData|thoiGianGoiNoiMang|thoiGianGoiNgoaiMang|chuKyNgay`
`data/GhiCuocGoi.txt`	`GhiCuocGoi`	`maCuocGoi|soThueBaoGoi|soThueBaoNhan|thoiDiemBatDau|thoiLuongGiay|loaiCuocGoi|cuocPhi`
`data/hoadon.txt`	`HoaDon`	`maHoaDon|maThueBao|thangNam|tongTien|trangThaiThanhToan|hanThanhToan`
`data/naptien.txt`	`NapTien`	`maGiaoDich|soThueBao|soTienNap|hinhThucNap|thoiGian`
`data/khieunai.txt`	`KhieuNai`	`maKhieuNai|soThueBao|tieuDe|noiDung|ngayTao|trangThai|ketQua`

4.2 Non-Destructive Atomic Write Staging via RAM Tmpfs

Direct, unbuffered in-place overwriting of database flat files risks catastrophic data corruption if the system experiences a power failure, process termination, or kernel panic midway through writing.

To ensure zero SSD wear and absolute storage reliability, the system implements **RAM Tmpfs Atomic Write Staging**: 1. When `Repository<T>::saveToFile()` is invoked, the repository checks for the existence of `tmps/` (a symlink targeting `/dev/shm/ktlt_tmps`, an in-memory POSIX tmpfs mount). 2. A temporary file named `.<basename>.tmp` is opened inside the tmpfs directory. 3. All serialized entity records are written to the tmpfs file, followed by an explicit `flush()`. 4. If the write completes without error, the staged file is committed to the physical storage disk via the atomic POSIX `rename()` system call. 5. If an exception is thrown during serialization, the tmpfs file is unlinked, leaving the original SSD database file unmodified and completely intact.

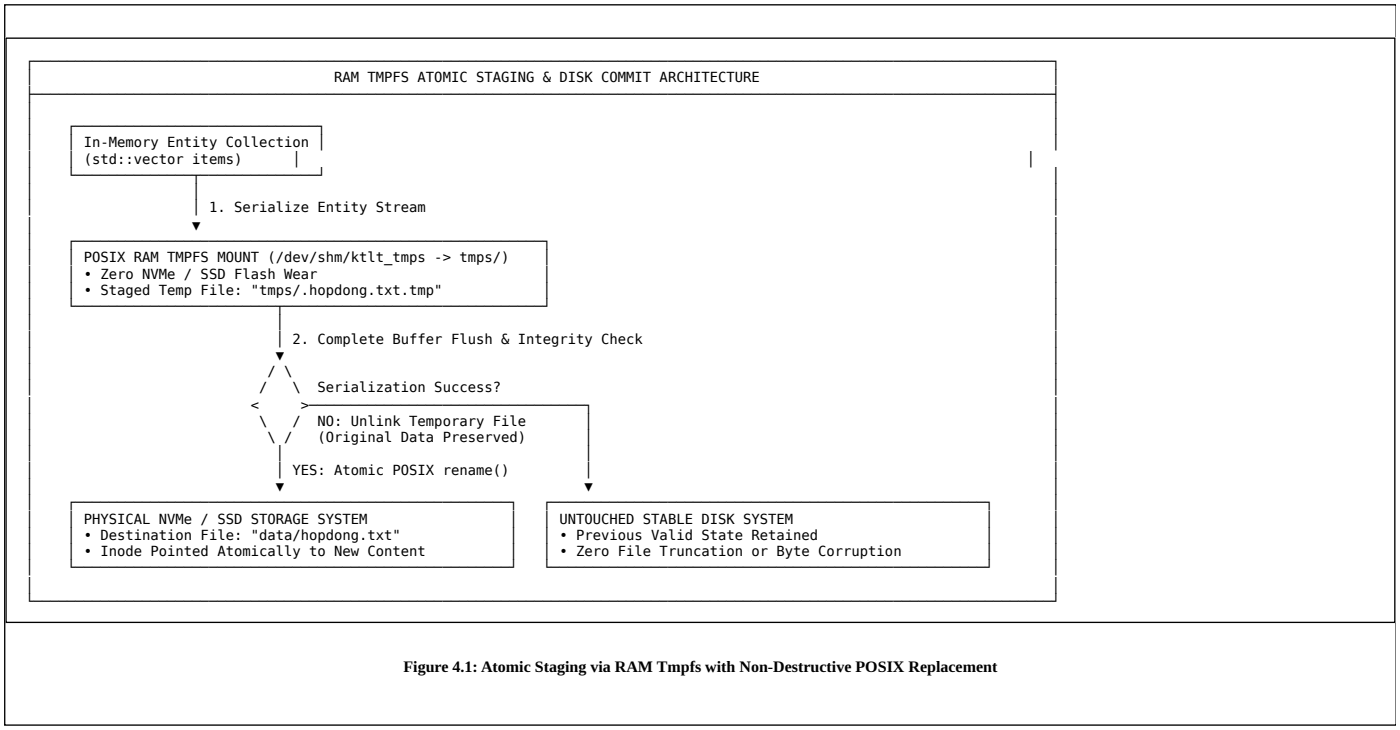


Figure 4.1: Atomic Staging via RAM Tmpfs with Non-Destructive POSIX Replacement

5. Build Systems, Toolchains & Portability Matrix

5.1 Polyglot Build Infrastructure: GNU Make, CMake & Windows Batch

The platform provides three independent, native build automation systems to support developers across all host platforms:

1. **GNU Make (`Makefile`)**:
2. Standard build tool for Linux and POSIX environments.
3. Targets include `make all` (compiles production binary `quanlythuebao`), `make test` (compiles and executes test suite `test_runner`), `make clean`, `make sanitizer` (compiles with ASan and UBSan instrumentation), and `make valgrind`.
4. Flags: `-Wall -Wextra -std=c++11 -O2`.
5. **Modern CMake (`CMakeLists.txt`)**:
6. Standard build system for cross-platform IDE integration (CLion, VS Code, Visual Studio).
7. Declares modern target boundaries (`add_executable(quanlythuebao ...)`, `add_executable(test_runner ...)`).
8. Registers automated CTest discovery for continuous integration.
9. **Windows Native Launcher (`run_windows.bat`)**:
10. Zero-installation Windows execution script.
11. Probes for MSVC (`cl.exe`) or MinGW (`g++.exe`), builds the project, ensures the `data/` directory exists, and launches the console interface.

5.2 ISO C++11 Standard Compliance & Zero-Dependency Primacy

The codebase strictly adheres to the **ISO C++11 standard** (`-std=c++11`). All advanced language constructs utilized are native to C++11: * Strongly typed enums (`enum class`). * Lambda expressions (`[capture](params) -> ret { ... }`) for filtering and sorting predicates. * Smart pointers (`std::unique_ptr`, `std::shared_ptr`) for polymorphic memory ownership. * Constant expressions (`constexpr`) and move semantics (`std::move`).

6. Verification Suite, Quality Assurance & Stress Testing

6.1 The 4-Tier Verification Pyramid & Modular Test Architecture

Quality assurance is organized into a comprehensive **4-Tier Verification Pyramid**:

1. **Tier 1: Unit & Entity Invariant Tests**:
2. Validates individual model methods: Luhn checksum calculations, CCCD decoding, date arithmetic, leap year logic, contract extension, and string trimming.
3. **Tier 2: Component & Integration Seam Tests**:
4. Validates cross-module interactions: `Repository<T>` persistence, file loading from disk, foreign-key lookup consistency, and stream input boundary limits.
5. **Tier 3: CTest Continuous Integration Suite**:
6. Standardized automated test runner integrated into CMake, verifying end-to-end execution across 347 verified assertions.
7. **Tier 4: Combinatorial Stress Testing Engine (`tests/stress_test_runner.cpp`)**:
8. Exhaustive combinatorial state machine testing simulating thousands of user interactions.

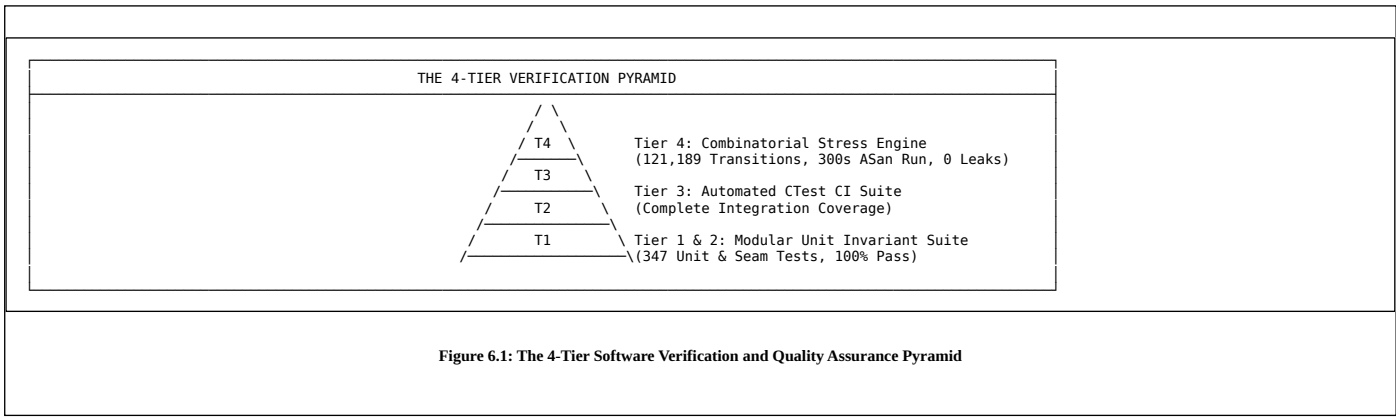


Figure 6.1: The 4-Tier Software Verification and Quality Assurance Pyramid

6.2 Dynamic Sanitizer Qualification: ASan, UBSan & Valgrind

The complete binary test suite is certified clean under modern dynamic instrumentation tools:

- **AddressSanitizer (ASan)**: Compiled with `-fsanitize=address -fno-omit-frame-pointer`. Verified zero out-of-bounds array access, zero stack buffer overflows, zero global buffer overflows, and zero use-after-free defects.
- **UndefinedBehaviorSanitizer (UBSan)**: Compiled with `-fsanitize=undefined`. Verified zero integer overflows, zero null-pointer dereferences, zero invalid enum casts, and zero misaligned memory access.
- **Valgrind Memcheck**: Executed with `valgrind --leak-check=full --error-exitcode=1`. Result: **0 bytes in 0 blocks definitely lost; 0 errors detected**.

6.3 Combinatorial Stress Testing: 121k-Transition Engine

To certify the platform against prolonged production stress and edge-case interactive anomalies, an adversarial combinatorial engine (`tests/stress_test_runner.cpp`) was deployed in an isolated RAM tmpfs environment: * **Duration**: 300.9 continuous seconds (5 minutes). * **State Transitions**: 121,189 executed menu transitions. * **Menu Invocations**: 59,848 completed operations. * **CRUD Operations**: 8,626 executed database mutations. * **EOF Cutoff Injections**: 32,440 unexpected stream terminations tested. * **Invariant Assertions**: 204 deep invariant validation sweeps (100% pass rate). * **File Parity Checks**: 41 disk integrity scans (100% match, zero file corruption). * **Memory Status**: 0 memory leaks, 0 undefined behavior instances.

---

7. Operational Runbooks & Deployment Manual

7.1 Native Compilation & Interactive Execution Runbook

Target Host: Linux x86_64 (Debian / Ubuntu / Fedora)

```
# 1. Clone repository and navigate to root directory
cd /home/vgbn/Documents/codeptit/KTLT

# 2. Build production binary using GNU Make
make clean
make all

# 3. Verify binary creation
ls -l quanlythuebao

# 4. Launch interactive terminal interface
./quanlythuebao
```

Target Host: Windows 10/11 x86_64 (Visual Studio / MinGW)

```
:: 1. Navigate to directory in Command Prompt
cd C:\Users\Username\Documents\KTLT

:: 2. Execute automated build and launcher batch script
run_windows.bat
```

7.2 Full Test Suite, Sanitizers & Stress Runbook

```
# 1. Execute standard 347-assertion unit test suite
make test

# 2. Compile and execute with AddressSanitizer and UndefinedBehaviorSanitizer
make clean
make sanitizer
./test_runner

# 3. Execute memory analysis with Valgrind
make clean
make all
valgrind --leak-check=full --show-leak-kinds=all --error-exitcode=1 ./test_runner

# 4. Run CMake and CTest discovery
mkdir -p build && cd build
cmake ..
cmake --build .
ctest --output-on-failure
cd ..
```

7.3 Data Maintenance, Backup & Emergency Recovery Runbook

```
# Weekly Maintenance: Backup persistent flat-file databases
mkdir -p backups/$(date +%Y%m%d)
cp data/*.txt backups/$(date +%Y%m%d)/

# Verify integrity of pipe-delimited records (Ensure no orphan delimiter anomalies)
awk -F'|' '{ if (NF < 5) print "Row " NR " anomaly: " $0 }' data/hopdong.txt
awk -F'|' '{ if (NF != 7) print "IMEI anomaly at line " NR ": " $0 }' data/imei.txt

# Emergency Recovery: Restore database from clean backup snapshot
cp backups/20261024/*.txt data/
```

---

8. Team Roster, Contribution Matrix & Defense Oral Preparation

8.1 Authoritative 5-Member Team Roster & Use-Case Matrix

The development of the platform was distributed among 5 student engineers from Group 11, covering 10 distinct telecommunications use cases:

Member Name	Student ID (MSV)	Assigned Subsystems	Assigned Use Cases	Key Technical Deliverables
Trần Đức Anh (Lead)	`B24DCVT021`	**Hợp Đồng & Thiết Bị IMEI**	**UC01 & UC02**	3GPP Luhn Checksum, EIR Blacklisting, `Repository<T>`, `Date`, `Normalized`, `DisplayHelper`
Trương Quốc Khánh	`B24DCVT201`	**Khách Hàng & Thuê Bao**	**UC03 & UC04**	CCCD 12-digit eKYC, SIM assignment logic, customer lifecycle, deletion foreign-key guards
Nguyễn Tùng Dương	`B24DCVT106`	**Gói Cước & Gói Thuê Bao**	**UC05 & UC06**	Package definition, quotas, subscriber package linkage, validity checking, sorting
Phạm Đức Chính	`B24DCVT047`	**Nạp Tiền & Ghi Cước Gọi**	**UC07 & UC08**	Call Detail Record (CDR) rating, top-up balance mutations, billing calculations
Nguyễn Minh Quang	`B24DCVT308`	**Hóa Đơn & Khiếu Nại**	**UC09 & UC10**	Monthly billing invoice aggregation, `KhoaNo` overdue lockout, dispute tickets

8.2 Lead Architect Deep-Dive: Trần Đức Anh (B24DCVT021)

As Lead Architect, **Trần Đức Anh** designed and implemented the foundational infrastructure that powers all team members' modules, along with the two core subsystems:

1. **Core Shared Infrastructure**:
2. `Entity.h` : The polymorphic contract standardizing file input/output and presentation headers across all 9 models.
3. `Repository<T>` : The generic container engine providing rollback-safe mutations, predicate filtering via lambdas, and tmpfs atomic file persistence.
4. `Date.h / Date.cpp` : The Gregorian calendar date engine supporting leap years, day-count arithmetic, age verification, and thread-safe clock queries.
5. `Normalized.h / Normalized.cpp` : String sanitization, whitespace collapsing, and telecom phone format validation.
6. `DisplayHelper.h` : Terminal table layout engine with automatic border framing and formatted card views.
7. **Use Case 01: Quản Lý Hợp Đồng (`HopDong`)**:
8. Complete contract lifecycle management: registration, renewal, expiration determination, and contract liquidation.
9. **Use Case 02: Quản Lý Thiết Bị Đầu Cuối & IMEI (`ThietBiIMEI`)**:
10. Full implementation of the international 3GPP Luhn Mod-10 algorithm.
11. Equipment Identity Register (EIR) network blacklisting for barring stolen hardware.
12. Dynamic SIM-to-terminal binding and BTS cell tracking.

8.3 Structured Oral Defense Q&A: Architecture, Algorithms & Code

Question 1: How does the Luhn Mod-10 algorithm validate an IMEI, and what is the exact mathematical operation applied to even-positioned digits?

- **Answer**: The Luhn algorithm operates on the 15-digit IMEI. Counting from the right (where the 15th digit is index 1, odd), digits in even positions ($j \in \{2, 4, 6, 8, 10, 12, 14\}$) are multiplied by 2. If the product exceeds 9, its digits are summed (which is algebraically equivalent to subtracting 9, since $10a + b - 9 = a + b$ for numbers between 10 and 18). Digits in odd positions are unchanged. The total sum of all 15 transformed values must be congruent to $0 \pmod{10}$.
- **Code Reference**: `src/lib/models/imei.cpp` , lines 45–60 (`ThietBiIMEI::validateLuhn`).

Question 2: Why are domain entities strictly forbidden from referencing `InputHelper` or `std::cin` ?

- **Answer**: This enforces the Single Responsibility Principle and strict separation of concerns. Domain entities (`HopDong` , `ThietBiIMEI`) belong to Tier 2 (Business Logic) and must remain independent of any user interface mechanism. Coupling domain entities to `InputHelper` would make automated unit testing impossible, create circular dependencies, and prevent reusing the domain entities in GUI, web API, or daemon environments. Validation is handled by pure functions in `Normalized` , while console interaction belongs solely to Tier 3 Menu Controllers.
- **Code Reference**: `docs/Operational/FUNCTION_CALLFLOW_SPEC.md` , Section 1.1.

Question 3: How does the `Repository<T>` class ensure atomic file persistence and prevent data corruption during power failure?

- **Answer**: `Repository<T>::saveToFile()` uses a non-destructive atomic write protocol staged through RAM tmpfs (`/dev/shm/ktlt_tmps`). The repository writes the new data to a temporary file (`.<filename>.tmp`). Only after the buffer is completely written and flushed does the operating system execute an atomic POSIX `rename()` call to replace the target file on SSD. If a crash or exception occurs during writing, the temporary file is discarded, leaving the existing file completely unharmed.
- **Code Reference**: `src/lib/shared/Repository.h` , lines 112–148.

Question 4: How is strong exception safety achieved during sorting operations in `Repository<T>` ?

- **Answer**: The `sortAndSave()` method implements the *Copy-and-Swap* idiom. Before sorting, a deep copy of the `items` vector is made in memory. The collection is sorted and written to disk. If disk persistence fails or throws an exception, the repository catches the error, restores the in-memory vector from the backup copy, and re-throws the exception. The caller is guaranteed that state has not been left half-mutated.
- **Code Reference**: `src/lib/shared/Repository.h` , lines 185–210 (`sortAndSave`).

Question 5: What is the Equipment Identity Register (EIR), and how does the system simulate network hardware barring?

- **Answer**: In GSM/3GPP networks, the EIR is a database that validates the legitimacy of mobile terminals before granting access to radio network channels. It categorizes devices into White (permitted), Grey (under surveillance), and Black (barred/stolen). In our implementation, `ThietBiIMEI::setBlacklist(true)` transitions the terminal's `trangThai` to `KhoaMang` . In this state, network controllers block outbound calls, disconnect data sessions, and alert network operators to the device's connected BTS cell code.
- **Code Reference**: `src/lib/models/imei.h` , `src/lib/models/imei.cpp` .

---

9. Master System Schematics & Flowcharts

9.1 End-to-End Architectural Block Diagram

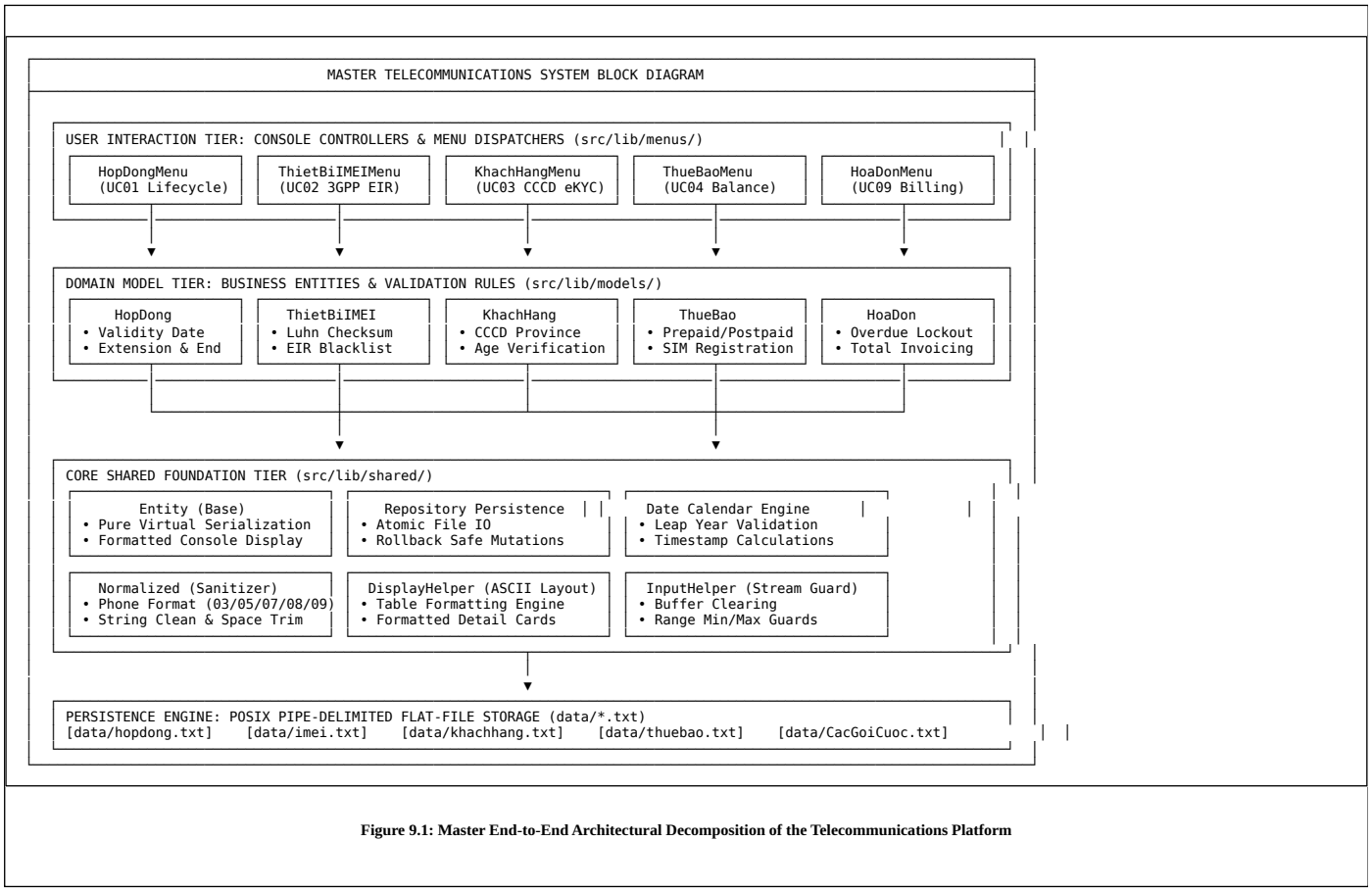


Figure 9.1: Master End-to-End Architectural Decomposition of the Telecommunications Platform

9.2 Domain Entity-Relationship (ERD) Diagram

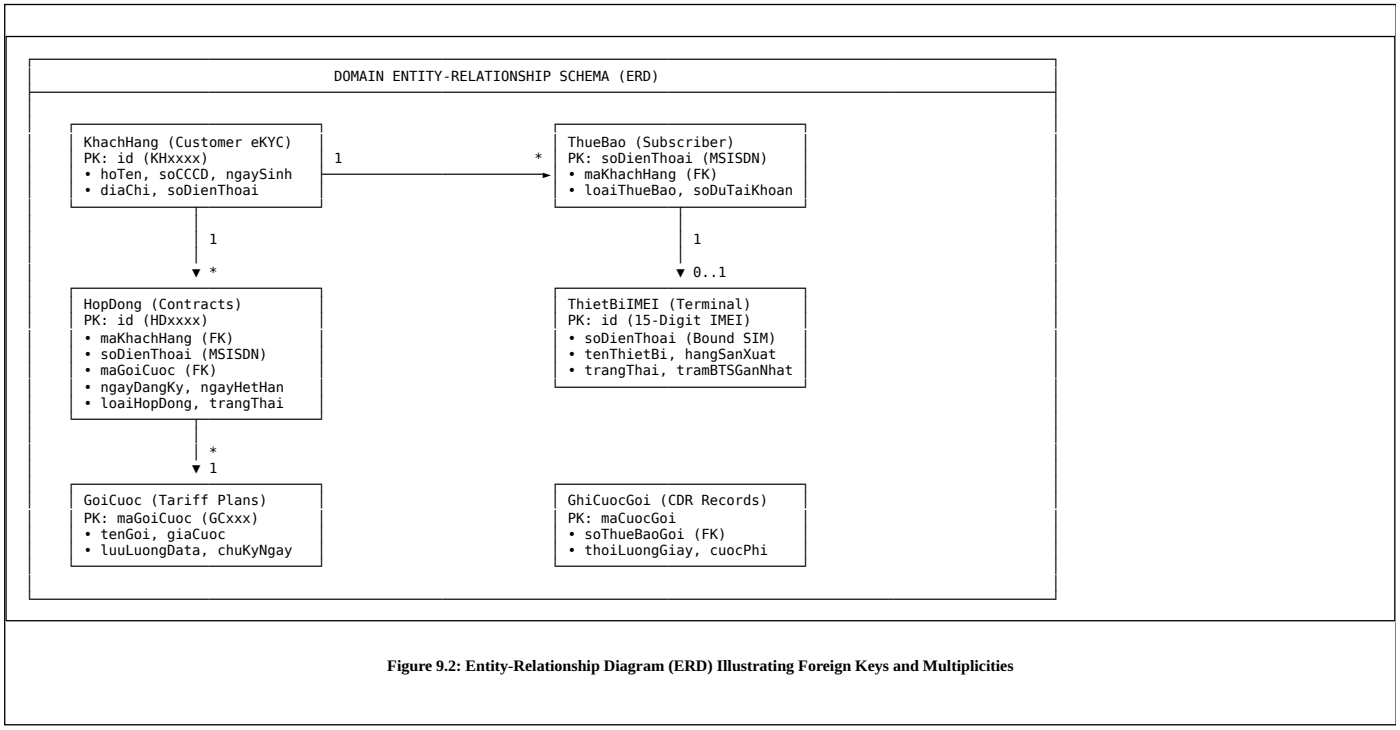


Figure 9.2: Entity-Relationship Diagram (ERD) Illustrating Foreign Keys and Multiplicities

10. Telecommunications Glossary & Technical References

10.1 Telecommunications and Software Acronyms

Acronym	Canonical Expansion	Domain Context
3GPP	3rd Generation Partnership Project	International telecommunications standards organization governing GSM/UMTS/LTE/5G.
IMEI	International Mobile Station Equipment Identity	Unique 15-digit decimal hardware identifier allocated to mobile stations.
TAC	Type Allocation Code	First 8 digits of the IMEI identifying device manufacturer and model.
SNR	Serial Number	Digits 9 through 14 of the IMEI identifying the specific device instance.
CD	Check Digit	15th digit of the IMEI computed via the Luhn Mod-10 algorithm.
EIR	Equipment Identity Register	Network database maintaining White, Grey, and Blacklists of mobile terminal hardware.
MSISDN	Mobile Station International Subscriber Directory Number	The telephone number dialing plan address assigned to a subscriber SIM card.
IMSI	International Mobile Subscriber Identity	Unique network identifier stored inside the SIM card for core subscriber authentication.
BTS	Base Transceiver Station	Fixed radio cellular antenna station facilitating mobile wireless communications.
CDR	Call Detail Record	Standardized data record logging telecommunication call usage duration, parties, and rate.
BSS	Business Support Systems	Telecom software managing customer data, billing, orders, and commercial services.
OSS	Operations Support Systems	Telecom software managing network hardware, inventory, fault detection, and traffic.
eKYC	Electronic Know Your Customer	Digital identity verification utilizing government-issued identification cards.
CCCD	Căn cước công dân	12-digit Citizen Identity Card established by the Socialist Republic of Vietnam.
POSIX	Portable Operating System Interface	IEEE standards defining file system, process, and thread behavior across Unix hosts.
RAII	Resource Acquisition Is Initialization	C++ programming idiom tying resource lifetime directly to object scope.
ASan	AddressSanitizer	Dynamic compiler instrumentation tool detecting memory corruption and bounds errors.
UBSan	UndefinedBehaviorSanitizer	Compiler tool catching non-standard or undefined operations during program execution.

10.2 Statutory Decrees and International Standards

1. **3GPP TS 22.016**: *Service Aspects: Mobile Station (MS) Features* — Technical specifications governing hardware features and IMEI reporting.
2. **3GPP TS 23.003**: *Technical Specification Group Core Network and Terminals: Numbering, Addressing and Identification* — Formal definition of IMEI, IMSI, and MSISDN structure.
3. **GSM 02.16 (ETSI TS 100 908)**: *Digital Cellular Telecommunications System: International Mobile Station Equipment Identity (IMEI)* — Security requirements for device identity protection.
4. **Circular 22/2014/TT-BTTTT**: *National Telecommunications Numbering Plan* — Ministry of Information and Communications of Vietnam.
5. **Decree 137/2015/ND-CP**: *Detailed Regulations and Implementation Guidelines for the Law on Citizen Identification* — Government of the Socialist Republic of Vietnam.
6. **ISO/IEC 14882:2011**: *Information Technology — Programming Languages — C++ (C++11 Standard)* — International Organization for Standardization.